

Your reusable storyboard desk

A beginner manual and asset catalogue for Blender and DaVinci Resolve. Version 1.0 • 1 October 2026.

Start with a short reviewed edit. Add exact text and deliberately selected objects. Preview one shot. Render a small batch only after you like the result.

This package preserves the basic book-reading and tutorial builders. The new overlay tools create separate editable scenes and transparent images. Your source recordings stay where they are.

The assets are generic and reusable. They are not an adaptation of The Brothers Karamazov, Orwell, Robert Greene or a history of democracy. Source meaning is UNASSIGNED until you select and review a passage.

Use this manual beside the screen. Complete one numbered step at a time. Save a new project before experimenting.

The package belongs under the installed blender-storyboard folder. Generated runs belong on E:, in Storyboard-Education-Runs. Keep the whole run folder, including its frames and receipt.

Contents

Choose what to use and when	3
First run: keep it small	4
What the basic edits preserve	5
Nine reusable assets	6
Asset catalogue: first five	7
Asset catalogue: four more	8
Manual Blender: import a GLB	9
Manual Blender: append the library	10
Manual Blender: edit readable text	11
Manual Blender: pose and animate	12
Overlay jobs: choose every input	13
Render transparent PNGs manually	14
Attach images to a Blender edit	15
Resolve: composite without Fusion	16
Objects behind you: optional manual work	17
Forty hours: build a ledger first	18
TypeSafe and source review	19
Troubleshooting and completion checks	20
Existing resources and verified results	21
References and next prompt	22

Choose what to use and when

Use this	When / resulting output
BOOK_ROUGH_EDIT.cmd	Rebuild the supplied book recording from reviewed cuts and 13 cues, then create a polished review copy. Includes the opening label and existing open-book inset.
TUTORIAL_ROUGH_EDIT.cmd	Rebuild the supplied tutorial from its reviewed cuts and captions, then create a polished review copy with words and timing preserved.
STATIC_OVERLAY.cmd	Build a separate stationary asset arrangement and editable text. Useful when you want a calm composition around a presenter.
ORBIT_OVERLAY.cmd	Build short animated movement inside side lanes. Useful after checking that moving objects do not distract from your reading.
stage_overlay.py	Use an explicit JSON job to choose static, side-entry or orbit mode, resolution, frame rate, text and asset files.
attach_overlay.py	Put a rendered PNG or sequence above a Blender edit and save a new project copy. Use a free strip channel.
plan_longform.py	Turn your timed shot CSV into a source ledger. It checks timing; it does not invent scenes.
typesafe_select.py	Request a bounded candidate judgment for a passage you selected. UNKNOWN means stop automatic semantic selection; explicit manual asset choices still work.

First run: keep it small

1. Open File Explorer with Windows+E. Open the installed blender-storyboard folder, then this education package.
2. Read README.md. If local-settings.json is missing, copy local-settings.example.json and rename the copy local-settings.json. In Notepad, set the Blender, source, plan and output paths. Keep both plan paths together. The installed local edition may already contain these settings.
3. Double-click BOOK_ROUGH_EDIT.cmd or TUTORIAL_ROUGH_EDIT.cmd. Read the console result. Do not start several Blender jobs at once.
4. Open the new run folder under E:/Resolve-Automation/Storyboard-Education-Runs. The timestamp identifies this attempt. Open review_edit.blend. The launcher retains rough_edit.blend as the historical draft and creates the corrected review copy plus a preview PNG.
5. If Windows asks which application to use, choose Blender. The installed executable is C:/Program Files/Blender Foundation/Blender 5.2/blender.exe.
6. Press Space to play, then Space to pause. Listen at cut joins. Read every subtitle. A saved project is a review edit, not a finished MP4.
7. Save a review copy with File > Save As. Give it a new name such as book_review_01.blend. Keep the generated JSON receipt beside the original output.

Checkpoint: you can hear the source, see the expected cut sequence, and find your saved copy. Continue only when this works.

What the basic edits preserve

The original builders live in `basic/`. Their source-bound plans compare the recording path, size, modification time and reviewed cut boundaries. They refuse a plan that belongs to another recording.

The launchers first preserve `rough_edit.blend`, then automatically run `restyle_captions.py` to create `review_edit.blend`. This corrects oversized legacy text-box margins and improves caption wrapping and contrast without changing words or cue timing.

The book build retains reviewed wording from the `original_text` column. It adds 13 cues and an opening label. The short existing inset uses the installed `A132-book-open.glb` asset; it is separate from the nine new EDU assets.

The tutorial build uses its own reviewed plan and caption receipt. Do not point its launcher at a new recording while retaining the old cut plan.

The builders save editable Blender projects. They do not recreate Resolve Voice Isolation or Fairlight EQ, and they do not render a final video automatically.

For a new recording, prepare a matching reviewed plan and captions, or run the underlying script without a cut plan to retain the complete recording. Supply an SRT when using the supported caption input. See `basic/README.md` for exact CLI arguments.

Before export: listen for abrupt words, check caption timing, check that labels cover no important material, and confirm the source audio remains synchronized.

Keep one untouched generated project and one review copy. This makes an experiment easy to undo without rebuilding everything.

Nine reusable assets

The contact sheet below shows the actual generated assets. EDU IDs remain stable when you change titles or source books. PNG is a rendered still; GLB is geometry; SVG is a simplified flat icon.

The geometry is original and the manifest declares CC0-1.0. Existing assets elsewhere in the storyboard installation retain their own provenance and licenses.



Asset catalogue: first five

EDU001 — Open book. A generic open volume. Use when you explicitly want a book object. Replace nearby title text yourself; no book identity is encoded.

EDU002 — Writing pen. A small writing instrument. Useful as a deliberately chosen prop or flat icon; it does not prove that a passage discusses writing.

EDU003 — Speech panel. A panel for short text. In the native Blender library, text stays editable. GLB text is fixed mesh geometry.

EDU004 — Theatrical mask. A generic mask object. No character, personality trait or hidden motive has been assigned to it.

EDU005 — Standing mirror. A freestanding mirror shape. Its reflective appearance can differ between renderers; inspect it in your target application.

For every asset, keep the ID with the shot record. Choose an object because you want it there, and record the source basis if it illustrates a passage. Do not treat a visually appealing object as evidence.

Asset catalogue: four more

EDU006 — Ballot box. A generic box and ballot. Select it explicitly for an appropriate shot; it does not represent every voting system or historical period.

EDU007 — Civic columns. A generic architectural grouping. It is not a reconstruction of a named institution, place or period.

EDU008 — Relationship nodes. A reusable node diagram. Add exact names and connections only after you have checked them against your source.

EDU009 — Pivot figure. A simple figure whose limbs have named object parents. It is not a skinned character rig, automatic actor, lip-sync system or walk cycle.

The assets have local origins and metre-scale dimensions recorded in `assets/manifest.json`. Keep imported parent relationships intact. Scale the root or a new outer parent instead of randomly scaling individual pieces.

For 2D layouts use SVG in a compatible vector application or PNG directly in Resolve. For editable 3D geometry use GLB in Blender. For editable native text use the `.blend` library.

Manual Blender: import a GLB

1. Open Blender. Choose General for a fresh scene. Save it with File > Save As to a new folder on E:.
2. If you want an empty scene, click the default cube, press X, then confirm Delete. Leave the camera and light until you know whether you need them.
3. Choose File > Import > glTF 2.0 (.glb/.gltf). Browse to this package, then assets. Select EDU001.glb. Click Import glTF 2.0.
4. Look in the Outliner at the upper right. Expand the imported tree. Select the top-level EDU001 root to move the complete object.
5. Move the mouse over the 3D Viewport. Press the decimal key on the numeric keypad to frame the selection. Without a keypad, use View > Frame Selected.
6. Press G to move, then X, Y or Z to constrain the axis. Type a distance and press Enter. Press S to scale. Press R to rotate. Esc cancels an unfinished transform.
7. Save with Ctrl+S. Test a small rotation, then Ctrl+Z to undo. Confirm all child pieces follow together.

Checkpoint: one whole object moves without leaving parts behind. If parts separate, undo and select the root parent.

Manual Blender: append the library

1. Save your current project. Choose File > Append. Browse to assets/education-library.blend and double-click it.
2. Open the Object section. Choose the objects you need, including their parent roots and associated children. If unsure, append the library scene through the Scene section into a temporary project first.
3. An easier first lesson is File > Open > education-library.blend, then immediately File > Save As to a new E: file. This keeps every object relationship and native text available.
4. Use the Outliner search field to find EDU003 or another stable asset ID. Expand its hierarchy to inspect the components.
5. Hide unrelated objects using their Outliner visibility controls while you work. Do not delete the installed master library.
6. To make an independent copy within a scene, select the complete root and child hierarchy before duplicating with Shift+D. Rename your copy clearly.

Why use native Blender? A GLB exchanges geometry reliably, but editable font objects and Blender-specific settings may not survive as the same native data. Use the .blend library when you want to edit the words.

Checkpoint: your title object is a Text object, and your file is a new copy on E:.

Manual Blender: edit readable text

1. Select a Text object in the Outliner or viewport. Overlay scenes name their text Editable title and Exact user caption.
2. Move the pointer over the viewport. Press Tab for Edit Mode. Press Ctrl+A to select existing text. Type or paste your exact replacement. Press Tab again to return to Object Mode.
3. In Properties, click the green text-data icon. Adjust alignment, size and spacing. Under Geometry, a small Extrude value gives real depth; zero keeps flat text.
4. Use the Material tab to change the colour. Keep strong contrast. Render a sample rather than judging readability only in the viewport.
5. Press numeric-keypad 0 for camera view, or choose View > Cameras > Active Camera. Check that the title fits fully inside the image.
6. Render a still with F12. Read it at the size you expect on a phone. If it looks cramped, shorten the card only with your own editorial approval, or split it into two cards.

The overlay builder fits text width geometrically; it does not proofread, rewrite, select key words, wrap a long paragraph into a lesson or verify spelling. Title limit: 100 characters. Caption limit: 160 characters.

Keep transcript wording in captions and use a separate label when you want your own commentary. That makes the distinction visible to viewers.

Manual Blender: pose and animate

1. Open a copy of the native library or import EDU009.glb. Expand the figure in the Outliner. Find a named limb object ending in `_PIVOT`.
2. Select the pivot, not only the visible limb mesh. Press R, then an axis key, then a small angle such as 15. Press Enter. Inspect which way it moves; undo if needed.
3. Set the timeline to frame 1. In the Item panel, right-click the Rotation property and choose Insert Keyframe. You can also insert keys through Object > Animation.
4. Move to frame 25. Change the pivot rotation slightly. Insert another Rotation keyframe on the same property.
5. Press Space to play. The limb should move between those poses. Stop and inspect frames 1, 13 and 25.
6. Open the Dope Sheet to see the keys. Move a key horizontally to change timing. Keep a saved copy before deleting keys.

This figure has parent pivots, not deforming skin, an armature or inverse kinematics. A limb moves as an object. It cannot automatically bend at an unmodelled joint or act out a selected passage.

The linked Grease Pencil tutorial teaches a different workflow involving drawing, rigging, weight painting and expressions. This pivot lesson does not claim to reproduce that full rig.

Overlay jobs: choose every input

A JSON job is a small text file containing explicit settings. Copy an example config before editing it. Keep commas, brackets and double quotes valid. Windows Notepad can edit it; save as UTF-8.

The launchers read configs/static.json, configs/side.json and configs/orbit.json respectively. Copy the whole package to a working folder before changing installed examples, or use a copied job with the script directly.

Required practical choices: asset paths, mode, width, height, fps, seconds, exact title and exact caption. Use the same fps as the receiving edit.

static holds objects still. **side** moves them into the side lanes over approximately half a second. **orbit** adds a small 3D movement inside those lanes.

subject_safe_rect is [left, bottom, right, top], measured from 0 to 1. Example [0.34, 0.16, 0.66, 0.88] reserves the centre area you explicitly specify. Inspect it against your actual recording.

The rectangle controls asset side lanes. It does not detect a face or body, track a camera, remove a background, or ensure that title/caption text avoids you. Review the complete composition.

Limits: 1–8 assets, 2–3000 frames, at most 120 seconds per shot, frame rates from 1–60. At 60 fps the 3000-frame cap means at most 50 seconds.

Use a new empty output folder. The builder refuses to replace an occupied one. Run --preview for one representative still; run --render for the full bounded PNG sequence.

Render transparent PNGs manually

1. Open overlay.blend from a new run. Save a review copy. Check the scene camera by entering camera view.
2. In Output Properties, confirm resolution, frame rate, Start and End. For a first test use 25 fps and frames 1–25.
3. Set the output folder to a new E: folder. Set File Format to PNG and Color to RGBA. The A channel stores transparency.
4. In Render Properties, open Film and enable Transparent. This removes the world background from the render; it does not remove your recorded room or extract your body.
5. Press F12 to render one frame. The background should show transparency in the render viewer. Check the title, object edges and lighting.
6. Choose Render > Render Animation, or Ctrl+F12. Blender writes numbered image files. Wait for the small test range to finish.
7. Open several frames from the output folder. Check the first, middle and last. Keep them together with their original numbered names.

A PNG sequence is not an MP4. It keeps alpha for compositing. Do not assume a normal H.264 MP4 preserves transparency. Compose first, then export the final movie.

Attach images to a Blender edit

Use scripts/attach_overlay.py with Blender in background mode. It opens your original project, adds an image strip and saves a different output file. It refuses to overwrite the source.

Arguments: --project points to the existing .blend; --images points to one PNG or a folder of overlay_*.png files; --output is a new .blend path; --start is the first timeline frame; --channel is an unused strip channel.

For one still, --still-seconds sets its duration (default 6). For a sequence, duration comes from the number of frames. The overlay must fit inside the existing edit.

The tool checks numbered sequence continuity. If a neighboring receipt exists, it checks frame rate. Do not mix frames from different runs or rename them casually.

Manual alternative: open your review project, switch to Video Editing, place the playhead at the start, choose Add > Image/Sequence, select the images, then place the strip above the footage. Set its blend mode to Alpha Over.

Check that the overlay is above footage but does not obscure important captions. Channel 6 is the script default, not a guarantee that the channel is empty.

Save the new edit and reopen it. Confirm that image paths still resolve before deleting or moving any run folders.

Resolve: composite without Fusion

1. Open Resolve and your review project. In the Media Pool, duplicate the timeline and give the copy a new name before experimenting.
2. Use the Media page to browse to the rendered PNG folder. In Media Storage options, ensure Show Individual Frames is off so numbered frames appear as a sequence. Import that sequence.
3. For a single static PNG, import the image directly. In the Media Pool, right-click the sequence and inspect Clip Attributes; confirm its frame rate matches the timeline.
4. Open the Edit page. Put the original footage on V1. Drag the transparent image or sequence onto V2 at the intended start time.
5. Select the overlay. In Inspector, use Transform Position and Zoom for final layout. Leave the normal composite mode unless you deliberately need another effect.
6. Play the first, middle and last frames. A solid background means the alpha settings or export need checking. Inspect Clip Attributes alpha handling if edges look wrong.
7. For a still, drag the end of the clip to set its duration. For animated sequences, retain the intended frame rate and check the full motion.

No Fusion composition is required for this transparent overlay workflow. Review audio and captions, then use Deliver to export your final composed video.

Objects behind you: optional manual work

The supplied overlay places objects in safe side lanes. It does not extract you from the recording or make objects automatically pass behind your body.

To show a genuine behind-body effect, you need a separate foreground layer containing only you with transparency. That requires a manual matte or another explicitly chosen extraction workflow.

The compositing order is: V1 original footage; V2 object overlay; V3 your extracted foreground. V3 covers the object where your body should be in front.

If you have a suitable manually prepared foreground PNG sequence, import it and align it exactly above the original footage. Check frame rate, starting frame and duration before judging the effect.

Creating the matte itself is a separate task. Resolve masking features and availability depend on edition and workflow. This package does not create the matte, track it, or certify an automatic subject-selection tool.

Inspect hair, hands and motion blur at full size. Scrub around movement. A weak matte often looks worse than a clean static side composition.

For a first book-reading video, use the static side layout. Add behind-body compositing only when you have reviewed the foreground mask and want the extra production work.

Forty hours: build a ledger first

Forty hours at 25 fps is 3,600,000 frames. Treat it as many reviewed shots and chapters. Do not launch a single giant render as your first test.

1. Choose the actual source edition and exact passage. Record a page, chapter, timestamp or another stable source reference. Keep the source text separate from public assets.
2. Create a CSV with these column names: `shot_id`, `start_seconds`, `end_seconds`, `source_ref`, `exact_text`, `asset_ids`. Put commas and multiline text inside CSV quoting as needed.
3. Give every shot a unique ID. Enter nonoverlapping ordered time ranges. Each shot must be no longer than 120 seconds. For 40 uninterrupted hours, that requires at least 1,200 maximum-length shots.
4. Run `plan_longform.py --csv your-shots.csv --output a-new-ledger.json --fps 25` with Python. It hashes the source CSV and each exact text field.
5. Review one short pilot: text, object choice, framing, pacing and audio. Mark `text_reviewed` and `visual_reviewed` only after you actually check them.
6. Render small batches. Record which shots completed and where their output lives. Resume from the ledger instead of guessing what was finished.

The planner preserves supplied text and timings. It does not write a screenplay, divide books into scenes or certify source coverage automatically.

TypeSafe and source review

TypeSafe is the semantic gate. It can return a bounded candidate asset judgment for a passage and candidate list that you supply. Geometry checks, timing checks and explicit IDs use deterministic code.

A live example selection succeeded with model jev-1.13.0 and returned EDU001. A separate design gate returned TEMPLATES_ONLY and CANDIDATES at confidence 1.0. These are bounded example/design judgments, not approval of literary or historical interpretations. The catalogue remains UNASSIGNED.

An earlier approved credential loader used a saved placeholder and returned UNKNOWN. A direct environment credential worked; existing configuration was not changed. Missing credentials still return UNKNOWN rather than a guessed choice.

typesafe_select.py takes --input and --output. Input JSON contains passage and assets; each candidate has id and description. It reads TYPESAFE_API_KEY from the environment and never needs the key in a job file.

A candidate result still needs review. Low confidence, no match, missing credentials and failures stay UNKNOWN. The script does not silently substitute another model or stage a scene from its result.

For Karamazov, retain the selected edition and the existing kit source rules. Do not replace the translation or assume a teaching demo covers a complete chapter.

For Orwell and Greene, use the actual selected passage. For history of democracy, identify the scope, period and source; one primary document is not a complete global history.

Keep exact quotations distinct from your commentary. Do not publish private source text, account references or credentials with an educational asset package. Public source links are listed at the end of this manual.

Troubleshooting and completion checks

The rough builder refuses the video: check the exact source path and matching review plan. A mismatch protects you from cutting the wrong recording. Do not disable the check to force it through.

The overlay refuses the output: choose a new empty run directory. **Asset too wide:** reduce its configured size or review the safe rectangle. **Too many frames:** shorten the shot.

Text is hard to read: enlarge it, increase contrast, simplify the composition or split cards manually. Render at delivery resolution and inspect at normal viewing size.

Parts move separately: undo, then select the parent root. **GLB words cannot be edited:** open the native .blend text object instead.

Images are missing after moving a project: restore their folders or relink paths. A .blend can refer to external recordings and images; copying only the .blend may be insufficient.

Before calling a shot complete: open the saved project, check text against the chosen source, inspect start/middle/end, listen to audio, confirm alpha and frame rate, then inspect the exported result.

This manual describes interfaces and observed reference access. It does not turn an unrendered scene into a verified final video. Consult each run receipt and the package validation report for actual completed checks.

Existing resources and verified results

The existing free_education repository already contains storyboard-factory-v3. This new package complements that collection and preserves its identity; it is not a replacement for the full existing library.

The v3 README lists 380 asset entries: 200 general objects, 128 Karamazov entries, 40 low-poly objects and 12 figure poses. It also lists 140 scene starts and a 56-page beginner PDF. Follow the README links for its existing downloads.

Public entry point:

https://github.com/sourovdeb/free_education/blob/main/storyboard-factory-v3/README.md

The older v3 verification covered portable assets. Native Blender execution was not established by that historical report. Keep historical claims separate from the new local checks.

The new basic launcher checks completed: book edit, 3 segments and 13 cues, 40.64 seconds; tutorial edit, 124 segments and 335 cues, 705.12 seconds. Static and orbit scenes were generated with previews. These are project/preview checks, not a finished long movie.

The launchers now run scripts/restyle_captions.py automatically after the preserved basic build. It produces review_edit.blend and a preview. A fresh book test preserved all 13 cues, words and timings while restoring the visible source brightness.

The correction sets text-box margins to 0.01 of image width; the old oversized margins covered the frame. Captions wrap and use stronger contrast. Segoe UI or DejaVu Sans is selected when installed, with the original font as fallback; no font file is bundled.

To restyle another project directly, use --project, --output and --preview with a new output path. Inspect the PNG and JSON. Keep rough_edit.blend alongside review_edit.blend. Consult the package validation report for the latest completed checks.

References and next prompt

Video descriptions read; videos not watched. Automatic caption endpoints returned empty responses. Playlist metadata was partly retrievable; complete viewing and order are not claimed.

SHIN LOGIC: Grease Pencil stick-figure tutorial. Chapters describe drawing, rigging, weight painting, expressions, Time Offset and animation. <https://www.youtube.com/watch?v=HqlhGYi3rpc>

Kazi Ahmed: Codex CLI and Blender MCP procedural scene tutorial. Description specifies a short hospital-corridor scene and camera move. https://www.youtube.com/watch?v=_J3H6Lfwxu0

Storyboard and Animatic / Blender Grease Pencil playlist:
<https://www.youtube.com/playlist?list=PLw86iS0BRh9MONE9EI6lKp4BJja3lTb-d>

Blender TextCurve API: <https://docs.blender.org/api/current/bpy.types.TextCurve.html>

Blender output manual: <https://docs.blender.org/manual/en/latest/render/output/properties/output.html>

Khronos glTF importer/exporter: <https://github.com/KhronosGroup/glTF-Blender-IO>

Blender MCP third-party project: <https://github.com/ahujasid/blender-mcp>. Its current README names mcp-for-blender and supports the legacy name. No installation changes are required by this manual.

Copyable task prompts and public source links are in PROMPTS.md and PUBLIC-SOURCES.json beside this PDF. Start with one reviewed six-second shot.